I. Introducing MMQP

This section introduces the Middleware Message Queuing Protocol (MMQP) [2], a lightweight Internet of Things (IoT) and Machine-to-Machine (M2M) communication protocol, designed to address the limitations of Message Queuing Telemetry Transport (MQTT) discussed in (Section III). While MMQP operates at the application layer and supports conventional Transmission Control Protocol (TCP) connections, it also defines an explicit mapping to multiplexed transports such as QUIC [29], [30]. The design combines topic-profile registration, compact control packets, hierarchical payload encryption, and deterministic inter-broker dissemination. These mechanisms are intended for applications such as Vehicle-to-everything (V2X) and Industrial Internet of Things (IIoT), where resource limits, intermittent connectivity, and the separation of control traffic from application data are relevant design concerns.

A. Messaging Pattern

Similar to MQTT, MMQP follows a publish/subscribe communication paradigm that enables loosely coupled interactions through a broker. Publishers register the topics on which they intend to produce messages, while subscribers declare the topics they wish to consume. The collection of these declarations forms the client's *topic-profile*. As illustrated in (Figure 10), a client first sends a `CONREQ` control packet, which MAY contain its initial profile, and receives a `CONACK` after successful admission. Subsequent profile changes use `PROREQ` and `PROACK`. A publisher then sends a `PUBREQ`, and the broker resolves its topic and identifies eligible subscribers according to their profiles, requested roles, and Quality of Service (QoS) policies.

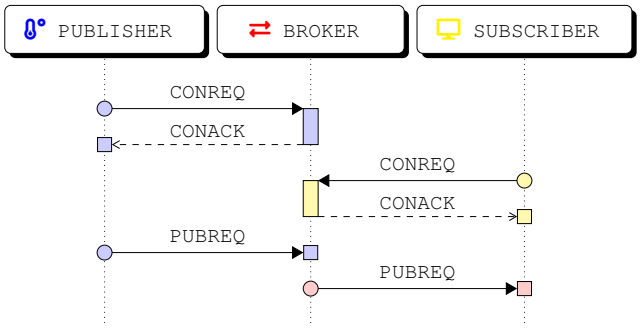


Fig. 1: MMQP Exchange Sequence

This explicit registration model allows the broker to validate publisher intent before accepting a publication. In a clustered deployment, profile declarations also provide the routing information used to select recipient sessions and construct the dissemination tree. Shared subscriptions and request/response interactions reuse this profile model, rather than introducing separate discovery protocols. More details on clustering, dissemination trees are provided in (Section VII).

B. Control Packets

(Table 4) summarises the nine control-packet types used by MMQP. Connection management, profile management, and publication each have their own request and acknowledgement operations, while `CONSYN` and `PROSYN` handle authentication/keep-alive and alias synchronisation, respectively. The smaller packet vocabulary reflects the selected protocol semantics: the implementation supports QoS 0 and QoS 1 with duplicate suppression, but does not implement MQTT's separate QoS 2 exchange or claim complete feature equivalence with MQTT.

Name	Flow	Description	Code
CONREQ	C → B	Connection request	0x10
CONACK	C ← B	Connection acknowledgement	0x20
CONSYN	C ↔ B	Authentication and keep-alive	0x30
CONEND	C ↔ B	Connection termination	0x40
PROREQ	C → B	Profile request	0x50
PROACK	C ← B	Profile acknowledgement	0x60
PROSYN	C ↔ B	Alias-mapping acknowledgement	0x70
PUBREQ	C ↔ B	Message, request, or response	0x80
PUBACK	C ↔ B	Publication acknowledgement	0x90

TABLE 1: MMQP Control Packets (C: Client, B: Broker)

1) Control Packet Structure: Each control packet consists of an always-present *fixed header*, followed by the packet-specific body. As shown in (Table 5), the first octet contains the Control Packet Type (CPT) in bits 7–4, the Remaining Length Flag (RLF) in bits 3–2, and the packet-specific Control Packet Flags (CPFs) in bits 1–0. The RLF selects the width of the following remaining-length field; the value of that field counts body bytes and excludes the complete fixed header.

Bytes	7	6	5	4	3	2	1	0
1	CPT				RLF		CPF	
w	Remaining-Length ($w = 1, 2, 3, 4$)							

TABLE 2: MMQP Fixed Header

All multibyte wire integers use network byte order. Text and token fields have a one-octet byte-length prefix unless explicitly specified otherwise, giving a maximum of 255 encoded bytes per such field. Text MUST be well-formed UTF-8. The decoder rejects unsupported packet types, reserved flag combinations, invalid identifiers, inconsistent lengths, and trailing bytes in typed bodies. Stream framing is incremental: a packet may span several network reads, and one read may contain several packets. In the packet tables (Table 5), the first column denotes the *size of each field*, rather than an absolute byte offset. Symbols such as ℓ_t denote the length declared by the preceding length octet. Optional fields occur only when their stated condition is satisfied. This notation remains valid when preceding blocks or remaining-length widths vary.

2) Remaining-Length Encoding: The RLF specifies a remaining-length width $w \in \{1, 2, 3, 4\}$. Its two-bit value is

$w-1$, placed in bits 3–2 of the control octet. The encoder selects the smallest width representing the body length L_R:

$$N_{\mathrm{RLF}}(L_R) = \begin{cases} 1, & 0 \leqslant L_R \leqslant 255, \\ 2, & 256 \leqslant L_R \leqslant 65{,}535, \\ 3, & 65{,}536 \leqslant L_R \leqslant 16{,}777{,}215, \\ 4, & 16{,}777{,}216 \leqslant L_R \leqslant 4{,}294{,}967{,}295. \end{cases}$$

The maximum representable body length is therefore $2^{32}-1$ bytes. This is an encoding limit, not an advertised device capability. Configured and negotiated packet-size limits are checked before allocating the advertised body, and a non-minimal remaining-length representation is rejected.

3) RLF vs. Variable Length Encoding (VLE): As introduced in (Section III-B), MQTT encodes the remaining length using seven data bits and a continuation bit per octet. Both schemes need one length octet below 128 bytes; MMQP also represents lengths from 128 to 255 in one octet. With four length octets, the respective maximum body values are $2^{32}-1$ and $2^{28}-1$ bytes. The MMQP decoder learns the complete length-field width from the control octet, whereas a continuation-based decoder determines it while reading the length field. Both procedures have a fixed upper bound of four length octets.

From a computational perspective, RLF decoding is significantly simpler and more predictable than conventional VLE-based approaches. The decoder processes the RLF and subsequently reads a fixed number of bytes, irrespective of the encoded value. In contrast, MQTT decoding requires iterative byte-by-byte processing until a termination condition is met (i.e., the continuation bit is set to zero), introducing conditional branching and increased parsing complexity. By eliminating the iterative decoding loop inherent to VLE, RLF provides constant-cost decoding, thereby reducing computational overhead and improving parsing efficiency.

C. Session Management

A client begins session establishment by transmitting CONREQ, as shown in (Table 6). Its body starts with the protocol alias "MMQP", protocol version 0x01, and the session-property flags. Optional properties appear in the order Session Authentication Flag (SAF), Session Policy Flag (SPF), QUIC Policy Flag (QPF), Session Will Flag (SWF), and Topic Profile Flag (TPF). Authentication, session policy, transport policy, and wills each have a local flags octet. The topic-profile block instead contains a length-delimited PROREQ body. This organisation groups related options without requiring every session to carry every property. Reserved bits remain invalid in the current version; introducing additional properties requires an agreed versioned extension.

1) Resume Control Flag (RES): The RES flag requests the resumption of an existing session. The client MUST include its Session-Identifier (SID) in the authentication-properties block, and the broker verifies that the session exists, has not expired,

Bytes	7	6	5	4	3	2	1	0
1	CONREQ (0x10)				RLF		RES	0
w	Remaining-Length ($w = 1,2,3,4$)							
1	Protocol-Length (0x04)							
4	Protocol-Alias ("MMQP")							
1	Protocol-Version (0x01)							
1	SAF	SPF	QPF	SWF	TPF	0	0	0
Session Authentication (SAF = 1)								
1	SID	TOK	0	0	0	0	0	0
1	(SID = 1) Identifier-Length							
ℓ_s	(SID = 1) Session-Identifier							
1	(TOK = 1) Token-Length (32)							
32	(TOK = 1) Authentication-Token							
Session Policy (SPF = 1)								
1	IRS	SEI	KAI	MPS	0	0	0	0
1	(SEI = 1) Encoded Session-Expiry							
1	(KAI = 1) Encoded Keep-Alive							
4	(MPS = 1) Maximum-Packet-Size							
QUIC Policy (QPF = 1)								
var.	Policy Block (Table 17)							
Session Will (SWF = 1)								
1	BWM	LWM	TOP	0	0	0	QoS	
1	(BWM = 1) Birth-Will-Length							
ℓ_b	(BWM = 1) Birth-Will-Payload							
1	(LWM = 1) Last-Will-Length							
ℓ_l	(LWM = 1) Last-Will-Payload							
1	(TOP = 1) Topic-Suffix-Length							
ℓ_t	(TOP = 1) Will-Topic-Suffix							
Topic Profile (TPF = 1)								
1	0	0	0	0	RLF		0	0
w_p	Profile-Body-Length ($w_p = 1,2,3,4$)							
var.	PROREQ Body, Including PID (Table 12)							

TABLE 3: CONREQ Packet; Optional Blocks Appear in Flag Order

and belongs to the same authentication realm. A fresh session receives a newly generated identifier in CONACK, as illustrated in (Table 7). The implementation generates 16 random bytes and encodes them as a 32-character hexadecimal SID; this identifies a session, rather than a permanent physical device.

Bytes	7	6	5	4	3	2	1	0
1	CONACK (0x20)				RLF		0	0
w	Remaining-Length ($w = 1,2,3,4$)							
1	Reason Code (RC)							
1	(IRS = 1, RC $\neq$ 0) Reason-Length							
ℓ_r	(IRS = 1, RC $\neq$ 0) Reason-String							
1	SAF	SPF	QPF	SWF	TPF	0	0	0
Session Authentication (SAF = 1)								
1	SID	0	0	0	0	0	0	0
1	(SID = 1) Identifier-Length							
ℓ_s	(SID = 1) Session-Identifier							
Other Selected Property Blocks								
var.	Same Block Layouts as (Table 6)							

TABLE 4: CONACK Control Packet Layout

Resumption restores the retained profile, directional alias maps, publication sequence state, receive checkpoints, and pending deliveries. Broker-side storage can preserve this state

across broker restarts, and a surviving client can resume against another configured endpoint after ownership transfer. The client must still retain its own session state: restarting the client executable creates a fresh SID. A fresh session is also required after publication-identifier exhaustion or an unsafe gap in a multiplexed receive sequence. The negotiated transport and QUIC policy remain fixed for a resumed session.

2) Session Authentication Flag (SAF): The SAF block carries an optional SID and authentication token, selected by the `SID` and `TOK` bits. Authentication uses HMAC-SHA256 with a separately provisioned shared credential K_{auth} of at least 32 bytes. This credential is distinct from the payload-encryption root secret introduced in (Section V); a broker does not receive the latter.

Bytes	7	6	5	4	3	2	1	0
1	CONSYN (0x30)				RLF		TOK	0
w	Remaining-Length ($w = 1, 2, 3, 4$)							
1	(TOK = 1) Token-Length (32)							
32	(TOK = 1) HMAC-SHA256 Response							
TOK = 0: Empty Body, Remaining-Length = 0								

TABLE 5: CONSYN Authentication or Keep-Alive Packet

The client generates a 32-byte token T and includes it in `CONREQ`. The broker returns its response in a token-bearing `CONSYN`:

$$R_b = \text{HMAC-SHA256}_{K_{\text{auth}}}(T). \tag{1}$$

After verifying R_b, the client computes:

$$R_c = \text{HMAC-SHA256}_{K_{\text{auth}}}(R_b), \tag{2}$$

and returns it in a second `CONSYN`. The broker verifies R_c before issuing a successful `CONACK`. Both responses contain the complete 32-byte HMAC. Comparisons use constant-time verification. If authentication is mandatory, a missing or invalid token terminates establishment. (Figure 11) illustrates the exchange, with H denoting HMAC-SHA256.

For multiple tenant realms, the first eight token bytes identify the realm through the leading bytes of SHA-256 over `"MMQP/AUTH/"` followed by its configured identity; the remaining 24 bytes are random. A token-replay record is retained for one hour and survives broker restart when storage is enabled. In a cluster, replay admission is coordinated through quorum-owned records. These checks do not introduce a fresh server challenge into the exchange. The default authenticated configuration therefore requires verified Transport Layer Security (TLS) or QUIC; the HMAC transcript alone is not a permanent freshness guarantee or a per-device identity service.

3) Session Policy Flag (SPF): The SPF block contains the reason-string preference, Session Expiry Interval (SEI), Keep-Alive Interval (KAI), and Max Packet Size (MPS). If Include Reason String (IRS) is enabled, every nonzero reason code is followed by a one-octet string length and the diagnostic text. Codes below `0x80` represent successful or

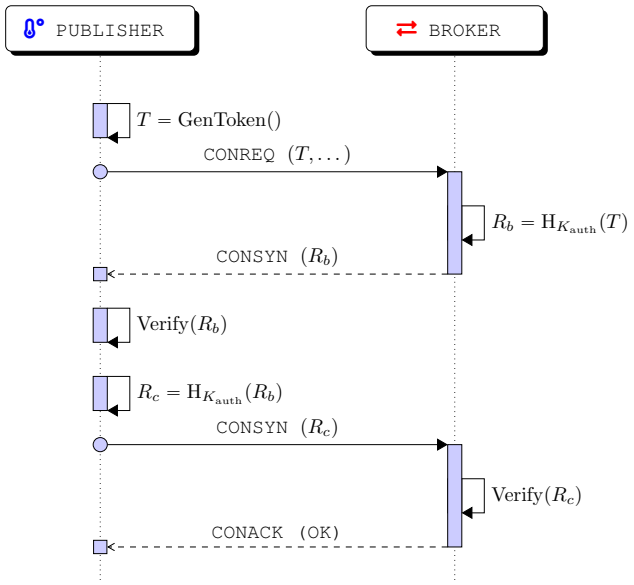


Fig. 2: Shared-Credential Authentication Exchange

informational outcomes, while codes greater than or equal to `0x80` represent failures. Code `0x00` has no reason string. This rule applies consistently to `CONACK`, `CONEND`, `PROACK`, `PROSYN`, `PUBACK`, and response publications. For example, the implemented registry assigns `0x01` to no subscriber, `0x02` to alias mapped, `0x8D` to no responder, `0x8E` to busy, and `0x8F` to timeout.

The SEI controls how long a disconnected session remains eligible for resumption. The MPS bounds the complete encoded packet accepted by an endpoint. A publication exceeding the negotiated limit is rejected; the current protocol does not fragment and reassemble application publications. Transport segmentation remains a separate operation.

The broker initiates the keep-alive exchange after the negotiated inactivity interval by sending an empty `CONSYN` with `TOK=0`. The client responds with the same packet, as illustrated in (Figure 12). A response deadline detects an unresponsive connection. This reuses the synchronisation packet without adding a dedicated ping packet type. The current broker does not initiate a new token-authentication exchange as part of an established-session heartbeat.

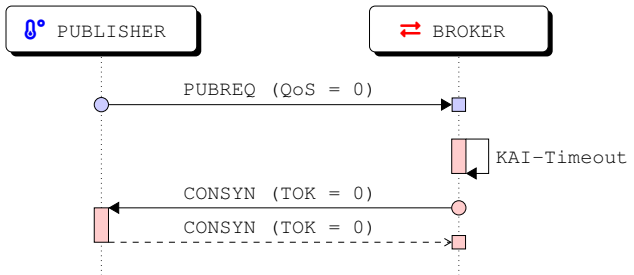


Fig. 3: Keep-Alive Handshake

a) Time Interval Encoding: The Tiered Interval Encoding (TIE) scheme encodes positive intervals in one octet, covering selected durations from one second to 63 days. As depicted in (Table 9), the upper two bits select a tier T, and the lower six bits contain a multiplier M. This trades uniform resolution for a compact representation. The comparison with MQTT's 16-bit field concerns Keep Alive specifically; it does not describe every time-related property in MQTT.

7	6	5	4	3	2	1	0
Tier (T)		Multiplier (M)					

TABLE 6: Encoded Time Interval Layout

T	B_T (sec)	Representable Intervals
0	1	1–63 seconds, in steps of 1 sec
1	60	1–63 minutes, in steps of 1 min
2	3,600	1–63 hours, in steps of 1 hour
3	86,400	1–63 days, in steps of 1 day

TABLE 7: Time Tiers and Supported Resolution

For $T \in \{0,1,2,3\}$ and $M \in \{1,\ldots,63\}$, the decoded interval is:

$$I = MB_T, \qquad V = 64T + M. \tag{3}$$

For instance, to encode an interval of 35 seconds, we would use $T = 0$, $M = 35$, and $V = 0\text{x}23$. The maximum is $63 \times 86{,}400 = 5{,}443{,}200$ seconds ≈ 63 days. Zero multipliers and unrepresentable durations are rejected. In particular, 64 seconds is not silently rounded to a different duration. The encoder selects an exact representation whenever one exists.

4) QUIC Policy Flag (QPF): The QPF block selects the QUIC session policy, optional bulk-stream limits, and optional datagram threshold. It provides a transport-specific binding around the same typed packet model. The transport motivation follows earlier work on QUIC [31]; the implemented mapping and its resource bounds are described in (Section VI).

5) Session Will Flag (SWF): The SWF block carries the Birth Will Message (BWM), Last Will Message (LWM), optional topic suffix, and publication QoS. The broker emits the BWM after a successful connection or resumption. A disconnection event, whether clean or unexpected, triggers the LWM once while that broker owns the session. Cluster fencing prevents an expired owner from continuing to issue session effects. Will topics use the reserved prefixes `"$bwm/"` and `"$lwm/"`. For example, the supplied suffix `"device/status"` produces `"$bwm/device/status"` and `"$lwm/device/status"`. If no suffix is supplied, the SID is used, with the tenant root prepended when configured. Ordinary root wildcards do not match reserved status topics; subscribers must explicitly name the reserved namespace. Will payloads use one-octet lengths and therefore remain bounded to 255 bytes each in this binding.

6) Topic Profile Flag (TPF): The TPF block embeds a complete PROREQ body, including its own profile Packet Identifier (PID), behind a separate remaining-length header. This permits session creation and initial profile admission within one establishment exchange. The result is reported through CONACK; the broker does not issue a second PROACK for this embedded request. Later profile changes use the normal PROREQ/PROACK exchange described in (Section IV-D).

7) Disconnection: Either endpoint may terminate a connection using CONEND, whose body contains a reason code and, when applicable, the negotiated reason string. (Table 11) shows the layout. A transport failure may prevent transmission of a final control packet, so connection cleanup also handles abrupt closure.

Bytes	7	6	5	4	3	2	1	0
1	CONEND (0x40)				RLF		0	0
w	Remaining-Length ($w = 1,2,3,4$)							
1	Reason Code (RC)							
1	(IRS = 1, RC ≠ 0) Reason-Length							
ℓ_r	(IRS = 1, RC ≠ 0) Reason-String							

TABLE 8: CONEND Control Packet

D. Profile Management

The PROREQ control packet consolidates SUBSCRIBE, REGISTER, and UNSUBSCRIBE operations. Several entries, including different operation types, may appear in one request. Each request has a nonzero 16-bit PID, followed by entries extending to the end of its body, as shown in (Table 12). The implementation validates the complete batch and its resource requirements before applying the profile changes; an invalid entry does not partially modify the local profile.

Every entry contains a topic alias. A concrete topic uses a nonzero alias, while a wildcard profile uses alias zero and carries its filter explicitly. The TOP bit indicates a one-octet length followed by the topic string. Once a concrete mapping is established, subsequent operations may identify it by alias alone. Profile-request identifiers form a sequence separate from publication identifiers, and PROACK identifies the request being acknowledged.

Topic are canonicalised by removing leading and trailing separators and empty hierarchy levels. The wildcard `"+"` occupies an entire level and matches one nonempty level; `"#"` occupies the final level and matches the remaining hierarchy. UTF-8 validation is mandatory, but case folding and Unicode normalisation are not performed. Consequently, endpoints must use the same canonical byte representation. Profile, alias, and session limits are explicit admission constraints.

1) Subscribe Request: A subscription defines the topic filter and accepted publication QoS. A QoS 0 subscription receives only QoS 0 publications, while a QoS 1 subscription accepts both implemented levels. The broker does not downgrade a QoS 1 publication to satisfy a QoS 0 subscrip-

Bytes	7	6	5	4	3	2	1	0
1	PROREQ (0x50)				RLF		0	0
w	Remaining-Length ($w = 1, 2, 3, 4$)							
2	Packet-Identifier (MSB First)							
Subscribe Entry								
1	SUB (01)		TOP	REQ	0	0	QoS	
2	Topic-Alias (MSB First)							
1	(TOP = 1) Topic-Length							
ℓ_t	(TOP = 1) Topic-Filter							
Register Entry								
1	REG (10)		TOP	REQ	0	0	QoS	
2	Topic-Alias (MSB First)							
1	(TOP = 1) Topic-Length							
ℓ_t	(TOP = 1) Topic-Filter							
1	Encoded Message-Expiry-Interval							
1	(QoS = 1) Maximum-Queue-Length							
Unsubscribe Entry								
1	UNS (11)		TOP	0	0	0	0	0
2	Topic-Alias (MSB First)							
1	(TOP = 1) Topic-Length							
ℓ_t	(TOP = 1) Topic-Filter							

TABLE 9: PROREQ Control Packet; Entries Repeat in the Body

Bytes	7	6	5	4	3	2	1	0
1	PROACK (0x60)				RLF		0	0
w	Remaining-Length ($w = 1, 2, 3, 4$)							
2	Acknowledged Profile PID							
1	Reason Code (RC)							
1	(IRS = 1, RC $\neq$ 0) Reason-Length							
ℓ_r	(IRS = 1, RC $\neq$ 0) Reason-String							

TABLE 10: PROACK Control Packet

tion. This is an intentional difference from MQTT's subscription delivery rules. Shared subscriptions use the form "$share/group/filter". The broker selects one eligible member for each distinct group/filter pair, including members connected to different brokers. Round-robin cursors belong to the ingress broker; they provide local selection fairness rather than a single cluster-wide scheduling order. Overlapping ordinary subscriptions are deduplicated by recipient session. A subscriber setting REQ also declares its ability to respond to requests on the matching topic, subject to request-specific responder admission.

2) Registration Request: Publishers register the topics or wildcard filters under which they intend to publish. In a REGISTER entry, the two-bit QoS field selects the *retention strategy*: zero retains the latest eligible message, while one maintains a bounded First-In-First-Out (FIFO) queue. This field does not cap the QoS of later publications. Only ordinary MESSAGE publications sent with QoS 1 are retained; QoS 0 traffic, requests, and responses do not enter the retained queue.

Each registration carries an exactly representable Message Expiry Interval (MEI). FIFO mode additionally carries a positive one-octet maximum queue length. When the configured queue is full, the oldest retained item is replaced; expiry removes items independently of that limit. Retained replay uses a separate bounded backlog and proceeds as acknowledgement credit becomes available, allowing a replay larger than the active in-flight window. An operation that cannot fit the configured replay or storage budget is rejected rather than silently truncating the replay. Setting REQ on a registration authorises the client to originate requests on matching topics.

3) Unsubscribe Request: An UNSUBSCRIBE entry removes the selected subscription state. A concrete entry references a known nonzero alias; a wildcard selection carries its filter with alias zero. The REQ and QoS bits are reserved and MUST be zero for this operation. Removing a subscription also updates the corresponding clustered profile snapshot, including an explicit empty replacement when no assertion remains in an affected shard.

4) Profile Synchronisation: The PROSYN control packet acknowledges a concrete alias mapping introduced through PUBREQ. It contains the alias, canonical topic, and outcome, as shown in (Table 14). A topic alias is scoped to a session and a direction: the publisher's outgoing namespace and the subscriber's incoming namespace are independent. It is not a cluster-wide numeric topic identifier.

Bytes	7	6	5	4	3	2	1	0
1	PROSYN (0x70)				RLF		0	0
w	Remaining-Length ($w = 1, 2, 3, 4$)							
2	Topic-Alias (MSB First)							
1	Topic-Length							
ℓ_t	Canonical Concrete Topic							
1	Reason Code (RC)							
1	(IRS = 1, RC $\neq$ 0) Reason-Length							
ℓ_r	(IRS = 1, RC $\neq$ 0) Reason-String							

TABLE 11: PROSYN Alias-Mapping Acknowledgement

Client-declared incoming aliases occupy $1 \ldots 32{,}767$, while the broker allocates concrete aliases for wildcard deliveries from $32{,}768 \ldots 65{,}535$. This separation avoids collisions between a profile update and an in-flight broker assignment. Publisher aliases may occupy the full nonzero 16-bit range. Existing mappings cannot be rebound to a different topic, and a sender continues to include the concrete topic until the receiver confirms the mapping.

E. Publishing Messages

A PUBREQ carries one of three publication types, selected by the two low control bits: MESSAGE=1, REQUEST=2, or RESPONSE=3. Every publication contains a nonzero topic alias. Its variable flags select retransmission (DUP), payload encryption (E2E), explicit topic (TOP), and QoS. A QoS 1 publication additionally contains its 16-bit PID. (Table 15) gives the complete field order, including the mutually exclusive request and response attributes.

The receiver resolves the canonical topic from the explicit string or the acknowledged directional alias map, checks the

Bytes	7	6	5	4	3	2	1	0
1	PUBREQ (0x80)				RLF		PRT	
w	Remaining-Length ($w = 1, 2, 3, 4$)							
2	Topic-Alias (MSB First)							
1	DUP	E2E	TOP	0	0	0	QoS	
1	(TOP = 1) Topic-Length							
ℓ_t	(TOP = 1) Canonical Concrete Topic							
2	(QoS = 1) Packet-Identifier							
Request (PRT = REQ)								
2	Correlation-ID (MSB First)							
1	Maximum-Response-Count							
Response (PRT = RES)								
2	Correlation-ID (MSB First)							
1	Response Reason Code (RC)							
1	(IRS = 1, RC $\neq$ 0) Reason-Length							
ℓ_r	(IRS = 1, RC $\neq$ 0) Reason-String							
Encryption (E2E = 1)								
4	Key-Epoch							
16	Public-Nonce							
16	Authentication-Tag							
Application Data								
var.	Payload (Plaintext or Ciphertext)							

TABLE 12: PUBREQ Packet; PRT-Blocks Are Mutually Exclusive

relevant profile and tenant policy, and admits the publication within its resource budget. The broker assigns independent outgoing identifiers and aliases for each subscriber session. The payload is forwarded unchanged, including its encryption envelope when present. A PUBACK acknowledges protocol acceptance at that hop, rather than completion of every downstream application action.

1) Request/Response: Request/response interactions reuse PUBREQ and operate with either supported QoS level. Requester and responder eligibility follows the REQ attributes in their profiles. A request includes a maximum response count; zero means all eligible responders subject to the broker's configured cap. A response contains its outcome and optional application payload. The broker returns an explicit no-responder result when no eligible responder is available, and outstanding requests have bounded deadlines.

To support concurrent exchanges on the same topic, requests and responses carry a two-byte Correlation-Identifier (CID) as depicted in (Table 15). A requester allocates an unused identifier in its own session, independently of the hop-specific publication PID. Applications may supply an explicit unused value, while the client otherwise allocates one automatically. The identifier is retained across retransmissions and session resumption. Values are not reused within the same SID, even after completion, rejection, or timeout, preventing a late response from being associated with a subsequent exchange. Exhaustion requires a fresh session, as does exhaustion of the corresponding responder-facing identifier space. A PUBACK still acknowledges hop-level acceptance through the PID; the response PUBREQ carries the application correlation.

The broker identifies an exchange by the requester SID and CID, and records its canonical topic, deadline, and admitted responder SIDs. Different requesters may consequently use the same numerical identifier. Before delivering a request to a responder, the broker assigns an unused identifier in that responder's session and records its mapping to the original exchange. The responder echoes the identifier it received, and the broker restores the requester's identifier on the return path. This mapping removes ambiguity when several requesters address the same responder on the same topic. A response is accepted only for the matching topic, an unexpired exchange, and a responder that has not already supplied its admitted response. Unknown, zero, or reused correlation identifiers are rejected. 0x90 is assigned to an invalid-correlation result. Broker-generated no-responder and timeout responses also carry the requester's identifier.

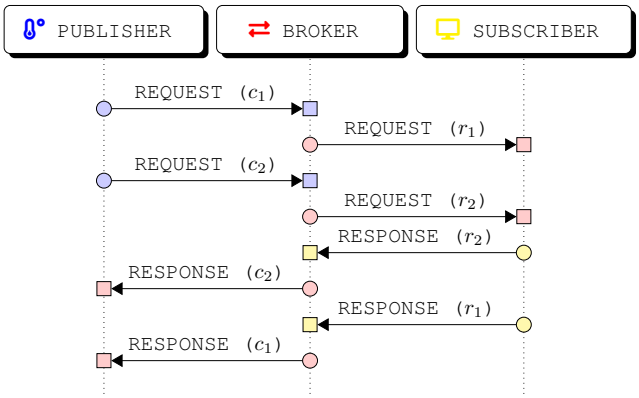


Fig. 4: Concurrent Request/Response Exchanges

(Figure 13) illustrates two overlapping exchanges. The requester uses c_1 and c_2, while the responder receives the independently assigned values r_1 and r_2. The second response may arrive before the first without requiring another topic or confusing the two calls. Session snapshots preserve the used requester identifiers, responder mappings, outstanding exchanges, and the remaining admitted responders after a partial response set. In a cluster, a quorum-owned directory is keyed by the requester session and correlation identifier. Its routing entry is updated when the requester resumes on another broker; the responder's mapping likewise follows its session state. For reliable clustered exchanges, the original publication is checkpointed until its incoming acceptance receipt is committed. After an interrupted admission, a successor repeats the same recipient-inbox writes before serving resumed traffic; the immutable message identity permits existing duplicate suppression to apply. Outstanding exchanges remain subject to request, queue, session, and storage limits. These mechanisms correlate protocol exchanges and do not make the responder's application effects transactional.

2) Delivery Guarantees & Retransmissions: MMQP implements two delivery levels. Both operate within configured queue limits, retry deadlines, and message/session expiry. The

PUBACK packet in (Table 16) contains the acknowledged publication identifier and its outcome.

Bytes	7	6	5	4	3	2	1	0
1	PUBACK (0x90)				RLF		0	0
w	Remaining-Length ($w = 1, 2, 3, 4$)							
2	Acknowledged Publication PID							
1	Reason Code (RC)							
1	(IRS = 1, RC ≠ 0) Reason-Length							
ℓ_r	(IRS = 1, RC ≠ 0) Reason-String							

TABLE 13: PUBACK Control Packet

- **QoS 0 (Unacknowledged Delivery):** The protocol sends a publication without a PID, PUBACK, or application-level retry. Delivery depends on the selected transport path and available resources. A QUIC datagram may be lost, while a stream supplies transport retransmission without changing the publication's QoS.
- **QoS 1 (Acknowledged with Duplicate Suppression):** A publication carries a monotonically increasing PID. An unacknowledged retry preserves that identifier and the canonical publication content, while setting DUP. The receiver recognises a previously accepted publication and suppresses repeated delivery within the session state.

The 16-bit publication sequence is monotonic within a session and direction. It MUST NOT wrap after 65,535. Admission reserves outgoing capacity before advancing the sequence, and exhaustion requires a fresh session. The DUP bit describes a retry; duplicate recognition is based on the identifier and its receipt, rather than on that bit alone.

a) Ordered Transports: For a single ordered TCP/TLS connection, the receiver maintains a high-water checkpoint S_x and a bounded receipt cache. A receipt binds a PID to the canonical publication content. A new identifier greater than S_x is eligible for admission; after acceptance, the checkpoint and receipt are updated. A repeated identifier is acknowledged without re-delivery only when its cached receipt matches. Conflicting content under the same identifier is a protocol error. An old identifier outside the receipt cache is rejected, rather than acknowledged on an unverifiable assumption. The checkpoint itself occupies constant space, but the complete mechanism also retains bounded receipts and pending delivery state. (Figure 14) illustrates a lost acknowledgement followed by a matching retry. Its simplified sequence omits storage operations; when persistence is enabled, the applicable checkpoint precedes the acknowledgement or outgoing delivery effect.

b) Multiplexed and Unordered Delivery: Ordering within one QUIC stream does not establish ordering across all streams in a session. We therefore use a 256-identifier receive window for QoS 1 on every QUIC allocation policy, including the single-stream policy. Let S_x be the highest contiguous accepted identifier and $W = 256$ the window width. The receiver applies the following rules:

1) A matching cached receipt identifies an accepted duplicate and permits acknowledgement without another delivery.
2) An unseen identifier satisfying $S_x < P_{id} \leqslant S_x + W$ may be admitted and marked in the bitmap after the required checks succeed.
3) An identifier above $S_x + W$ is rejected. The receiver does not advance the window past missing identifiers merely to accommodate later traffic.
4) As contiguous positions become complete, the receiver advances S_x and shifts the bitmap. Conflicting receipts and unverifiable stale identifiers remain errors.

Send admission also prevents the sequence from moving 256 positions beyond the oldest unacknowledged identifier. If an allocated identifier expires and leaves an unsafe gap, the client establishes a fresh session instead of attempting an ambiguous resume. The bitmap occupies 32 bytes, in addition to checkpoints, receipts, and queued publications. QoS 0 datagrams do not carry a PID and do not participate in this receive window.

c) Idempotent Publishing: The implemented form of idempotent publishing suppresses repeated protocol delivery while its receipt/checkpoint state remains valid. Broker persistence and clustered quorum checkpoints extend that state across supported recovery scenarios. They do not make application callbacks transactional. A crash between recording consumption and performing an external business action can still leave the action incomplete; replaying an action outside such a transaction can also duplicate its effect. Consequently, QoS 1 is not described as durable exactly-once application processing. Applications requiring that property must commit their side effect and consumed-message identity in the same application transaction.

3) Publisher Alias Mapping: After registering a wildcard such as "home/#", a publisher introduces a concrete topic such as "home/temp" by including its nonzero outgoing alias and the complete canonical topic in PUBREQ. The broker checks the registration and returns PROSYN. Until this confirmation arrives, subsequent publications retain the TOP field. The exchange in (Figure 15) permits later alias-only packets without making their interpretation depend on unconfirmed state.

4) Subscriber Alias Mapping: For a first delivery matching a previous wildcard subscription, the broker allocates an incoming alias in the range $32{,}768 \ldots 65{,}535$ and includes the concrete topic-string. The subscriber confirms it with a PROSYN control packet (Table 14), after which the broker may omit the topic string. (Figure 16) illustrates this reverse-direction mapping. A wildcard filter alone is insufficient to identify an encrypted message's concrete topic; the established alias map supplies that information on subsequent deliveries.

II. Lightweight End-to-End Encryption

The decoupled nature of publish/subscribe communication allows clients to exchange messages without establishing di-

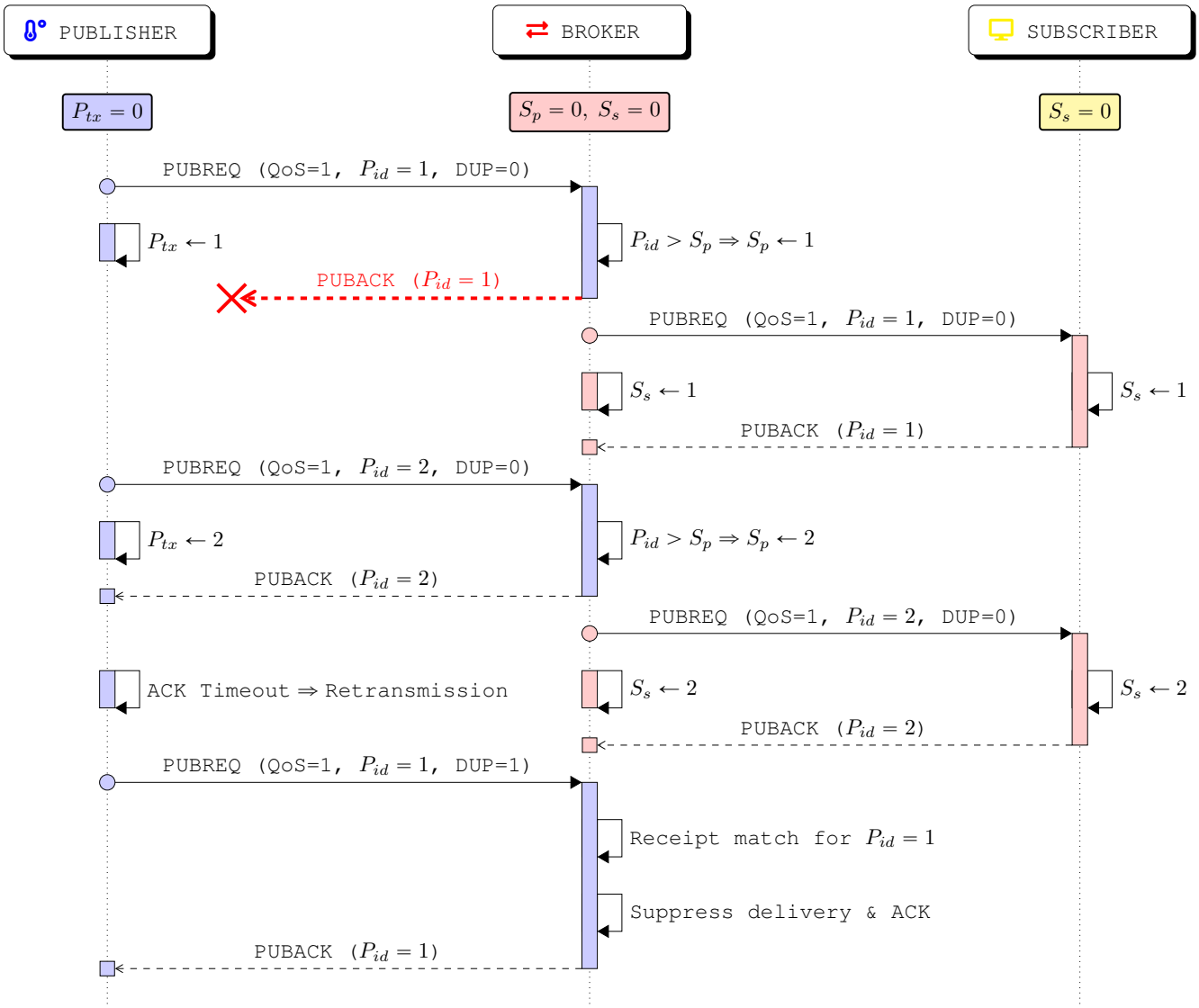


Fig. 5: Receipt-Checked Retransmission on an Ordered Transport

rect connections to one another. However, independently securing the publisher–broker and broker–subscriber links does not protect application contents from the broker itself. MMQP therefore separates transport protection from payload protection: TLS or QUIC secures each network connection, while client-side authenticated encryption protects the application payload across broker forwarding and retention.

A. Design Objectives

The proposed encryption architecture derives shared payload keys from the *topic hierarchy* and an externally provisioned trust anchor. Publishers and subscribers can consequently protect and recover a message without a direct client-to-client key exchange. A grant for a topic subtree permits derivation of its descendant keys, supporting wildcard subscriptions while preserving the broker's ability to route on visible topic metadata. The broker forwards the encrypted envelope and does not need the payload key or plaintext. The security objective is confidentiality and integrity of the payload, together with authentication of its canonical topic and key epoch. Successful authentication establishes possession of the corresponding symmetric key; it does not identify a particular device among several clients sharing that key. The distinction is relevant when a subscriber receives a subtree key, since that key also permits the creation of valid ciphertexts within the same grant.

B. Standardised Ascon Cryptography

MMQP uses **Ascon-AEAD128 as standardised in NIST SP 800-232** [32]. It uses a 128-bit key, a 128-bit public nonce, and a 128-bit authentication tag. The reference implementation is vendored from the Ascon team's published source [33],

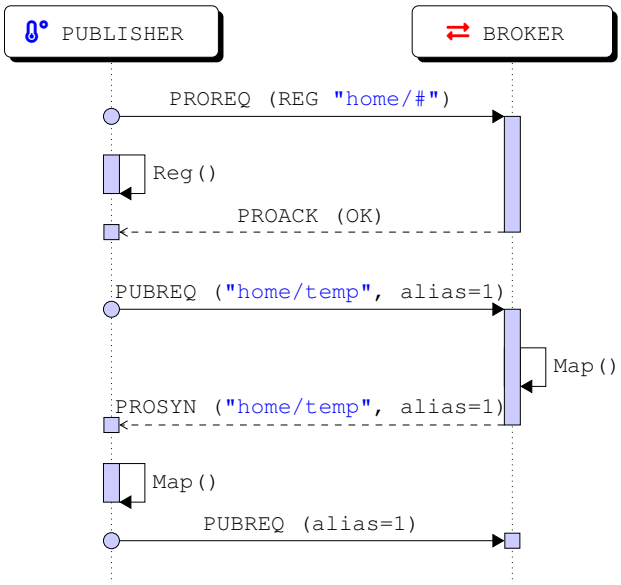


Fig. 6: Publisher Alias Mapping Acknowledgement

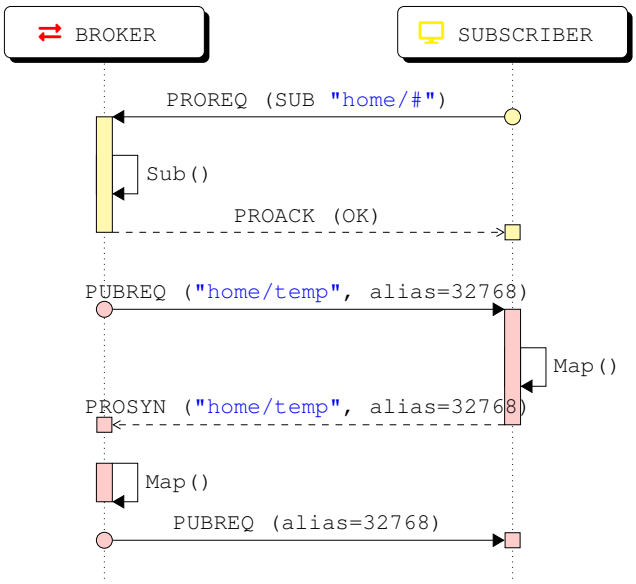


Fig. 7: Subscriber Alias Mapping Acknowledgement

with its revision and licence retained. The encryption and authenticated decryption operations are written as:

$$\begin{aligned}(C, H) &= \mathrm{Encrypt}(k_{\tau,E}, N, P, A),\\ P \text{ or } \perp &= \mathrm{Decrypt}(k_{\tau,E}, N, C, A, H),\end{aligned} \tag{4}$$

where P is the plaintext, C the ciphertext, N the nonce, A the associated data, and H the tag. The failure symbol $\perp$ indicates rejection without releasing candidate plaintext.

C. Trust Model & Key Provisioning

For payload confidentiality, the broker is not trusted with application secrets. It still controls admission, routing, retention, and availability, and can observe the cleartext metadata needed for those operations. A trusted *Key Provider* provisions either a domain root secret or an epoch-specific subtree grant to each authorised client. Provisioning occurs outside MMQP; the provider does not participate in ordinary message forwarding. The implementation accepts a 16- or 32-byte root secret and 16-byte delegated topic keys. Grants identify their authorised root and, for delegated keys, the applicable epoch. A centralised Key Management System (KMS) or hardware-backed provider can supply these grants, but that provider integration is not part of the current framework. Broker authentication credentials, inter-broker authentication keys, and payload secrets remain separate cryptographic domains. In a cluster, the brokers are trusted participants in the control plane even though they do not need access to encrypted contents.

D. Key Generation & Epochs

Let the canonical concrete topic be $\tau = l_1/l_2/\ldots/l_d$, and let E be a 32-bit key epoch. The implementation specifies HKDF-SHA256 [34] without leaving its salt or context encoding implicit. At each level, the parent key is the input keying material, the salt is the four-byte big-endian epoch, the information string is the exact UTF-8 representation of that level, and the output length is 16 bytes:

$$\begin{aligned}k_0 &= K,\\ k_i &= \text{HKDF-SHA256}\big(k_{i-1}, \mathrm{BE}_{32}(E), \mathrm{UTF8}(l_i), 16\big),\\ k_{\tau,E} &= k_d.\end{aligned} \tag{5}$$

For the namespace `"home/room1/temp"`, the successive keys correspond to `"home"`, `"room1"`, and `"temp"`. A client provisioned with the key for `"home/room1"` at epoch E can derive descendant keys for that epoch, but cannot derive its parent or sibling keys (Figure 17). The wildcard expression describes the authorised subtree; wildcard characters are never used as levels in a concrete publication key. Clients enforce a configured accepted-epoch interval and reject epochs outside the grant. Advancing the epoch generates a new hierarchy for holders of the root secret. A delegated epoch-specific key does not provide the next epoch's subtree key, so renewing a delegated grant requires another provisioning operation. Epoch rotation limits the scope of a leaf-key exposure, but does not revoke a compromised root secret or provide forward secrecy against its later disclosure.

E. Payload Encryption

For a publication on τ = `"home/room1/temp"`, the publisher derives $k_{\tau,E}$ and encrypts the payload. The `E2E` bit selects an envelope consisting of the four-byte epoch, 16-byte nonce, 16-byte tag, and ciphertext. These fields follow any routing and request/response attributes, in the order shown in (Figure 18). The ciphertext has the same length as the plaintext; the explicit encryption envelope adds 36 bytes before the ciphertext. The complete encoded packet must still fit the negotiated MPS. The broker can retain and replay this

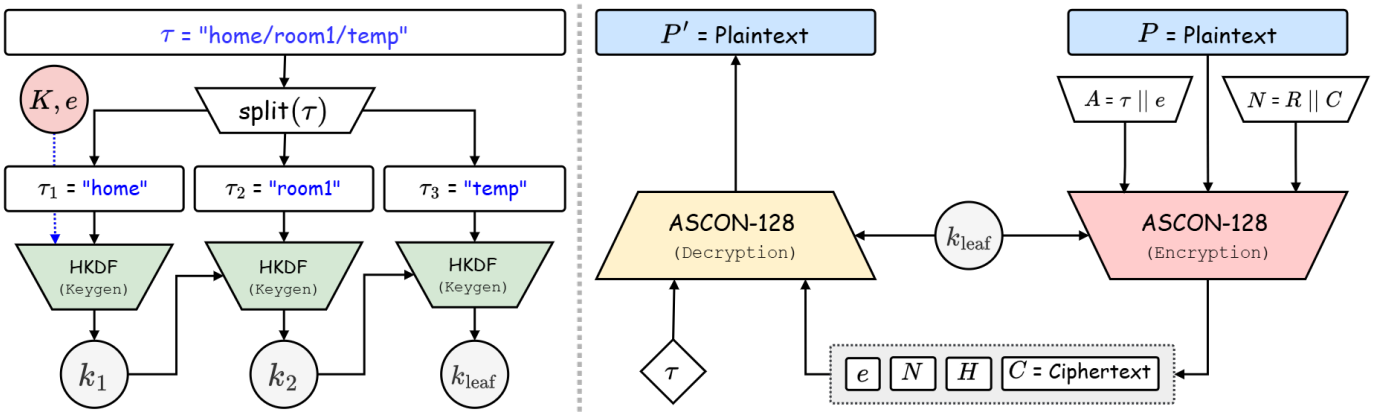


Fig. 8: Hierarchical Key Generation and Encryption/Decryption Procedure

envelope without decrypting it. Topic aliases and hop-specific publication identifiers may change during forwarding, while the canonical topic, epoch, nonce, tag, and encrypted payload remain the information needed for end-to-end verification. A retransmission reuses the original encrypted envelope rather than encrypting a different message under the same nonce.

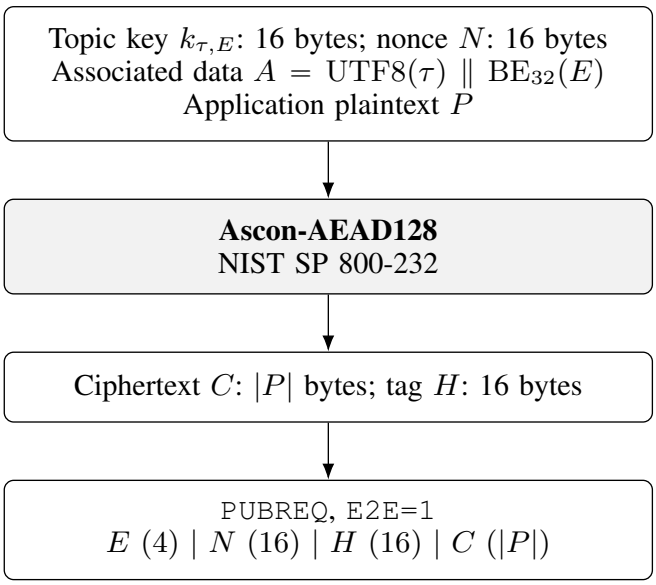


Fig. 9: PUBREQ Encryption Attributes (Full Layout in (Table 15))

1) Nonce Generation: The nonce combines a random publisher prefix with a monotonic counter:

$$N = R_{64} \parallel \mathrm{BE}_{64}(C_{\mathrm{nonce}}) \tag{6}$$

The prefix is generated from a cryptographically secure random source when a new cipher instance is created without existing state. The counter advances for each encryption and is rejected at exhaustion. With optional persistent cipher state, the same prefix and the next unused counter are restored after restart. Each counter is durably reserved before its ciphertext is released, so a crash may skip a counter but does not deliberately reuse one. The random prefix provides probabilistic separation among publishers sharing a key. For m independently selected prefixes, the approximate prefix-collision probability is

$$p_{\mathrm{coll}} \approx 1 - \exp\left(-\frac{m(m-1)}{2^{65}}\right)$$

It is therefore inappropriate to claim deterministic nonce uniqueness across all devices merely because a random prefix is used. Deployments requiring that stronger property need coordinated publisher-prefix allocation or an equivalent provisioning rule. Persistent cipher directories must not be cloned or rolled back.

2) Associated Data: Is defined byte-for-byte as:

$$A = \mathrm{UTF8}(\mathrm{canon}(\tau)) \parallel \mathrm{BE}_{32}(E) \tag{7}$$

The epoch occupies the final four bytes; no topic-length prefix is included in this associated-data value. Changing the canonical topic or epoch causes authentication to fail. All endpoints therefore canonicalise topics before key derivation and associated-data construction, removing leading/trailing separators and empty levels while preserving the remaining UTF-8 bytes. This binding covers the concrete topic and epoch, not every protocol field. In particular, the PID, publication type, response-count field, aliases, and other control flags are not included in A. The transport and broker admission rules protect those hop-level operations within their trust model. Replaying an unchanged envelope on the same topic and epoch also leaves its tag valid; replay suppression is a separate client-side state mechanism.

F. Authenticated Decryption & Key Caching

The subscriber resolves the concrete topic from the explicit topic field or its confirmed incoming alias map, verifies the epoch policy, and derives the required leaf key. For a wildcard delivery, the initial concrete-topic mapping is essential; the subscription filter itself is not a substitute for that topic. The

candidate plaintext is released only after the complete authentication tag has been verified. To avoid repeated hierarchical derivation, the client caches

$$(\tau, E) \longmapsto k_{\tau,E}$$

within a configured bound, which defaults to 128 entries. Evicted keys and cached keys removed during rotation or destruction are wiped. The cache reduces repeated derivation work, but its impact on latency and energy has not yet been benchmarked on constrained devices. After authenticating an envelope, the client updates a 256-counter replay window for its (τ, E, R_{64}) tuple. A known duplicate is not emitted to the application again. An unseen envelope older than the retained window is rejected, while a newer authenticated counter advances the window. Unlike the QoS 1 receive window, this cryptographic window may discard older unseen positions; it enforces an explicit replay/age policy rather than reliable delivery of every nonce value. Replay tuples are bounded, with a default limit of 4,096, and capacity exhaustion is rejected instead of forgetting live replay state to admit new identities.

When `"payload_state_directory"` is configured, nonce reservations, replay windows, and monotonic epoch policy survive restart. The directory is exclusively locked and bound to the key grant through an authenticated identifier, without storing the key itself. A replay checkpoint is written before plaintext is released. A failed state write makes the cipher unavailable until reopened. During retained replay, an already consumed encrypted envelope may arrive under a new broker PID; it can be acknowledged without delivering the plaintext a second time.

G. Security Considerations

The implemented security boundary follows from the key hierarchy, authenticated envelope, and maintained client state. These mechanisms protect application content without hiding the metadata used for routing.

1) Master Key Isolation: The current software implementation keeps its supplied grant in process memory. A root grant consequently exposes the hierarchy if the client process is compromised. A narrower delegated grant reduces that scope. Storing a root secret in an Hardware Secure Module (HSM) or equivalent protected provider is a deployment extension; the present code does not establish that only derived keys enter operational memory.

2) Temporal Containment: Disclosure of a leaf key compromises that topic and epoch, while a delegated subtree key compromises its authorised descendants for the same epoch. Accepted-epoch bounds can reject obsolete traffic and persistently prevent policy rollback. However, a retained root secret can derive historical and future epoch keys. Rotation alone therefore does not provide forward secrecy or recovery from root-secret compromise.

3) Metadata Exposure: The broker observes topic names, aliases, publication types, payload lengths, key epochs, nonces, timing, and communication patterns. It can also drop, delay, or reorder traffic within the limits imposed by the surrounding transport. Payload encryption provides neither traffic-analysis resistance nor an availability guarantee.

4) Integrity of Protocol Metadata: The authentication tag binds the payload to the canonical topic and epoch. It does not authenticate a distinct publisher identity or all request/response metadata. A holder of the same symmetric grant can produce valid publications. Applications needing per-device signatures or stronger metadata binding require an additional, explicitly versioned application or protocol envelope. Persistent replay suppression records cryptographic consumption, not completion of an application transaction. In particular, a crash after that record is committed but before an application callback finishes can prevent re-emission without guaranteeing the business effect. The same distinction between protocol state and application effects applies to the QoS mechanism.

III. MMQP over QUIC

MMQP natively exploit the capabilities offered by modern transport protocols such as QUIC [30] for scenarios requiring advance message transportation. TCP has several design characteristics that are increasingly misaligned with the requirements of contemporary IoT systems. It exposes communication as a single bidirectional byte stream and guarantees reliable, in-order delivery of all data. While these properties simplify application development, they also introduce performance limitations. Because all application traffic is multiplexed onto a single logical stream, the loss of a single segment prevents subsequent data from being delivered to the application until the missing segment has been retransmitted and received. This phenomenon, commonly known as Head-of-Line Blocking (HoLB), increases end-to-end latency and can significantly degrade throughput in lossy wireless environments. Furthermore, TCP provides no native mechanism for connection migration. This migration is common in V2X and IIoT systems. Changes in a device's network attachment point typically require the establishment of a new transport connection, introducing additional signalling overhead and disrupting ongoing sessions.

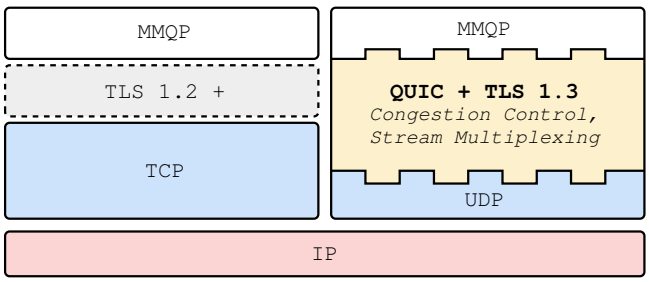


Fig. 10: MMQP over TCP vs. MMQP over QUIC

Standardised in 2021 by the Internet Engineering Task Force (IETF) (RFC 9000), QUIC is a User Datagram Protocol (UDP)-based multiplexed and secure transport protocol which integrates congestion control, loss recovery, native stream multiplexing, and TLS cryptographic handshake directly into the transport layer (Figure 19). By supporting multiple independent delivery streams, QUIC eliminates TCP's HoLB for independent message paths (Figure 20)., while simultaneously reducing connection establishment latency. Its ability to migrate connections across changing networks further enhances robustness, particularly in mobile and intermittently connected environments. These characteristics make QUIC a suitable transport substrate for MMQP, especially in deployments requiring scalability, mobility support, or stringent latency guarantees. Unlike MQTT, which treats the transport layer as a simple byte pipe, MMQP natively maps its application semantics onto QUIC primitives.

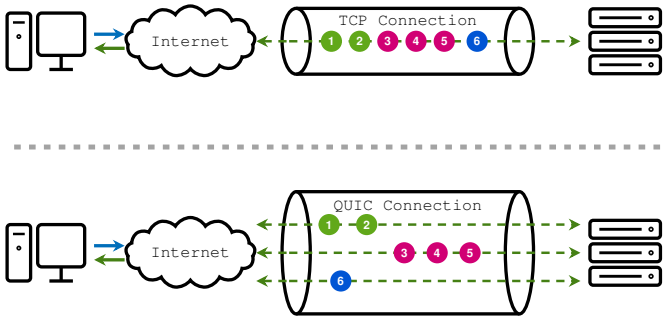


Fig. 11: TCP vs. QUIC Stream Operation

A. QUIC Policy Negotiation

The client proposes its transport policy through the QPF block in the CONREQ control packet. The broker accepts supported values or returns bounded alternatives in CONACK. Negotiation selects a stream-allocation strategy, optional bulk thresholds/concurrency, and optional datagram threshold. These parameters remain fixed for that session. A transport or policy change requires a fresh session rather than silently changing the interpretation of outstanding publications.

1) Stream Allocation Strategy: Assigning one transport stream to every topic would couple transport resource use to a potentially large and changing namespace. MMQP instead groups publications by communication class. The conceptual classes in (Figure 21) are bound to the following streams:

- **Control stream:** The client opens bidirectional stream ID zero. It carries session establishment, keep-alive, profile changes, alias synchronisation, disconnection, and acknowledgements. Under the single-stream policy it also carries publications.
- **QoS 0 semantic stream:** Under the semantic policies, each endpoint may open a persistent unidirectional stream for QoS 0 publications. Stream delivery remains reliable even though the application protocol does not acknowledge these publications.
- **QoS 1 semantic stream:** Each endpoint may open a second persistent unidirectional stream for acknowledged publications. Session-level duplicate suppression remains necessary across reconnects and application retries.
- **Bulk stream:** Under the bulk policy, an eligible large publication may use a temporary unidirectional stream, subject to the negotiated per-direction concurrency limit. One complete publication is followed by FIN.

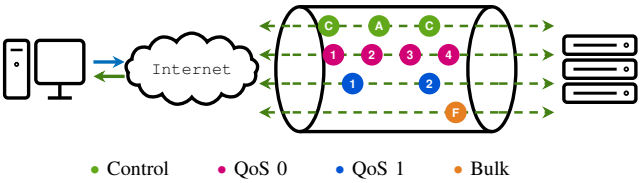


Fig. 12: Classes of Streams

Each unidirectional semantic or bulk stream starts with a one-octet class marker: 1 for QoS 0, 2 for QoS 1, or 3 for bulk. Ordinary MMQP frames follow that marker. These class markers and the control-stream convention are explicit reference-binding choices, not information implied by a generic QUIC connection. The implemented strategy values retain the original design's assignments:

- **0x01:** One bidirectional stream for all MMQP traffic.
- **0x04:** One bidirectional control stream and two persistent unidirectional publication streams per sending endpoint.
- **0x03:** The semantic mapping above, augmented with bounded temporary bulk streams.
- **0x00:** Reserved for an application-defined allocation policy. The generic implementation rejects it because no interoperable custom mapping has been specified.

The number of persistent streams therefore does not grow with the number of topics. Frames arriving on non-control channels before CONACK completes are held only within a bounded pre-negotiation queue. The binding accepts no 0-RTT app-data.

Bytes	7	6	5	4	3	2	1	0
1	SAS	BTS	QDG	0	0	0	0	0
1	(SAS = 1) Allocation-Strategy							
1	(BTS = 1) Maximum-Bulk-Streams							
4	(BTS = 1) Bulk-Payload-Threshold							
4	(QDG = 1) Datagram-Payload-Threshold							

TABLE 14: QUIC Policy Block within CONREQ or CONACK (QPF = 1)

2) Bulk Transfer Streams: The Bulk Transfer Stream (BTS) policy contains a positive maximum bulk-stream count and a payload-size threshold. A publication meeting that threshold may use a separate stream if concurrency credit is available. Otherwise, it falls back to the appropriate reliable semantic stream without changing its QoS. Bulk streams

carry complete publications, so they do not bypass the negotiated packet-size limit or add application fragmentation. Send buffers remain owned until the native transport reports completion, including during cancellation and shutdown.

B. Unreliable Datagrams

The QUIC Datagram (QDG) policy permits eligible QoS 0 publications to use the unreliable datagram extension defined by `RFC 9221` [35]. A datagram contains one complete `PUBREQ` and no semantic-stream class prefix. Datagram loss is not repaired by a stream retransmission mechanism, although the traffic remains subject to QUIC's connection-level congestion control and packet protection. A topic string or encryption envelope may therefore make an otherwise small payload ineligible. Oversized datagrams fall back to the reliable publication stream while retaining QoS 0. This provides an explicit path for transient telemetry without claiming that datagrams always reduce latency or increase throughput under every load.

C. Migration vs. Resumption

Transport migration and application-session resumption are distinct operations. If a new path is validated and the connection is kept alive, the existing MMQP session remains attached to that connection. If the connection terminates, the client opens another transport connection and performs `CONREQ` with `RES` and its retained SID. The automatic reconnection can rotate through configured failover endpoints, subject to session expiry and ownership rules (Section VII). Migration support does not imply uninterrupted operation for every network handover. The current validation establishes native communication and reconnect behaviour; it does not qualify a mobility matrix covering changing interfaces, path Maximum Transmission Units (MTUs), carrier networks, and middleboxes.

D. Middlebox Interference

As a UDP-based protocol, QUIC traffic may be aggressively rate-limited, timed out, or blocked by legacy enterprise firewalls and Network Address Translations (NATs). Although adoption has significantly improved due to HTTP/3, restrictive network environments remain a deployment consideration. To mitigate UDP blocking by restrictive firewalls or NATs, MMQP over QUIC could mandates the use of Application-Layer Protocol Negotiation (ALPN) over the standard UDP port 443. By registering a dedicated identifier, the protocol aligns with HTTP/3's deployment model and reuses the same cryptographic handshake profile. This approach maximises compatibility with modern network perimeters while avoiding custom firewall exceptions but neither port 443 nor an ALPN identifier guarantees passage through an arbitrary firewall.

IV. Hyperscale Deployment

While a single broker can efficiently handle millions of concurrent connections, geographically distributed deployments, high-availability guarantees, and massive IoT ecosystems demand more. To achieve this, our framework must transition from a broker-centric model toward a clustered architecture. This section extends the base design with mechanisms that ensure loop-safe dissemination, bounded state propagation, and failure-resilient session management, while preserving the lightweight nature of the protocol.

A. Clustering Strategies

Clustering brokers involves separate choices about physical connectivity, subscription placement, forwarding, and storage. Several approaches can satisfy parts of these requirements, with different operational costs. Loop freedom is a property of the forwarding rules applied to a topology, rather than a consequence of having more or fewer physical links.

1) Fully Meshed Clusters: In this topology, each of n brokers has a direct connection to every other broker, giving $n(n-1)/2$ undirected interconnections. (Figure 22) illustrates this structure. Direct reachability simplifies next-hop selection, particularly for small clusters. If every broker also replicates every subscription, however, each node bears the global profile-state cost. Broadcasting every publication additionally consumes bandwidth at nodes without interested subscribers.

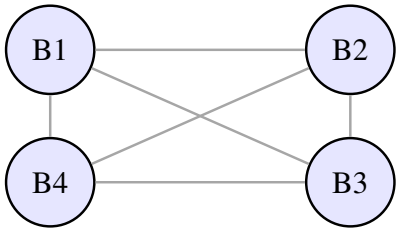


Fig. 13: Fully Meshed Architecture

A mesh does not inherently require broadcast or cyclic forwarding: selective routing and explicit message identities can limit both. Its main trade-off is the cost of maintaining dense connectivity and any globally replicated state chosen by the implementation. These costs should be distinguished from the payload-forwarding cost of a particular publication.

2) Hierarchical Topologies: Hierarchical architectures organise brokers into edge, regional, and core layers, as illustrated in (Figure 23). Traffic can be aggregated near devices and forwarded only along relevant branches. A directed tree provides a simple acyclic dissemination rule and allows routing policy to reflect deployment locality.

The hierarchy also concentrates traffic and availability dependencies at aggregation nodes. Capacity planning must account for those shared paths, and reconfiguration must preserve the relationship between routing domains and physical deployment. This is only useful when locality is known, but does not by itself solve session replication or durable recovery.

3) Distributed Log-Based Architectures: A distributed event log, such as a Kafka- or Pulsar-based backbone, can

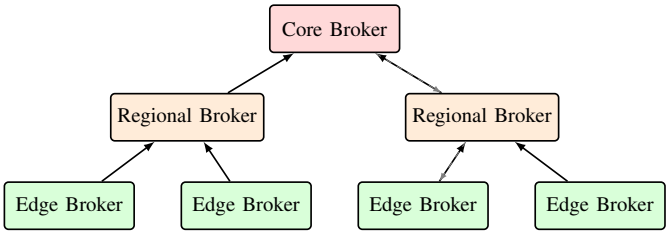


Fig. 14: Hierarchical broker topology with edge, regional, and core layers

separate device-facing brokers from durable storage and downstream consumption. In this arrangement (Figure 24), access brokers act as protocol gateways, while the event infrastructure supplies its own partitioning, retention, and consumer model.

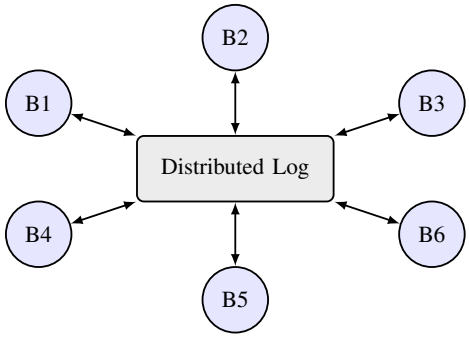


Fig. 15: Distributed log-based broker architecture

This separation enables replay and independent processing services and introduces another operational layer that requires a precise mapping between client acknowledgements and log persistence. End-to-end ordering, duplicate suppression, and application transactions still depend on the complete path; they are not obtained solely by inserting a log between brokers.

4) Gossip and Flood-Based Architectures: This overlays distribute information through repeated exchanges with selected peers. Flood-based schemes forward more broadly, usually with duplicate suppression or hop constraints. (Figure 25) shows one possible dissemination path within such an overlay.

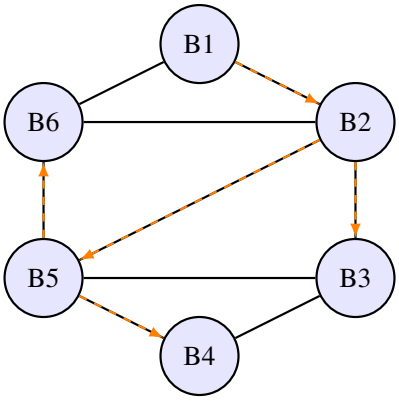


Fig. 16: Gossip and flood-based publish/subscribe architecture

5) Hybrid Architecture: This arrangement combines hierarchical access brokers with an event-storage backbone, as illustrated in (Figure 26). This can preserve traffic locality while providing a separate substrate for replay, analytics, and independent consumers. The architecture is one possible composition of the preceding mechanisms, rather than a universal deployment requirement. Such a deployment must still define message identities, acknowledgement boundaries, and the state restored after failure. Our approach uses similar separation of responsibilities, but implements recipient selection and session recovery within the native broker framework. An external analytical log is not required to run the implemented cluster.

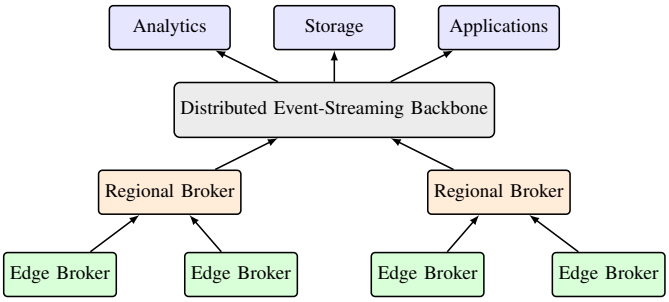


Fig. 17: Hybrid architecture combining hierarchical broker layers

B. Native Hyperscale Fabric

These approaches can accommodate decentralised discovery and changing connectivity. Their forwarding cost and convergence behaviour depend on fan-out, retry policy, duplicate tracking, and the failure model. They may be appropriate for membership or anti-entropy tasks even when the application payload path uses deterministic routing. For MMQP, we select an explicit recipient tree for ordinary inter-broker dissemination rather than making payload delivery depend on probabilistic flooding. The MMQP Hyperscale Fabric (MHF) combines explicit topic-profile registration with deterministic broker placement. A publisher's access broker queries the relevant profile shards, selects eligible recipient sessions, and constructs a bounded dissemination tree over their access brokers. Session replicas preserve the state needed for ownership transfer, while persistent recipient inboxes recover accepted QoS 1 work if an immediate forwarding path fails. The MHF arrangement follows five design objectives:

- **Controlled routing state:** Concrete profiles are assigned to logical shards. Wildcard profiles remain visible wherever their concrete matches may occur, with that replication cost accounted for explicitly.
- **Deterministic loop freedom:** Each accepted forwarding record carries enough information to reconstruct an ingress-rooted tree and validate every parent/child hop.
- **Session survivability:** A single leased owner serves a session, while a selected replica majority stores its recovery state and fencing information.

- **Transport-aware dissemination:** Mutually authenticated QUIC separates control and data classes; an explicitly configured mutual-TLS backplane supports deployments without QUIC.
- **Explicit durability boundaries:** Reliable ingress and retained state use bounded persistent records. Transient QoS 0 traffic does not acquire a durable-delivery promise merely by entering a cluster.

1) Architectural Overview: The logical architecture retains four cooperating planes, as illustrated in (Figure 27). They describe responsibilities rather than four separately deployed services; a broker process may perform several roles according to deterministic placement.

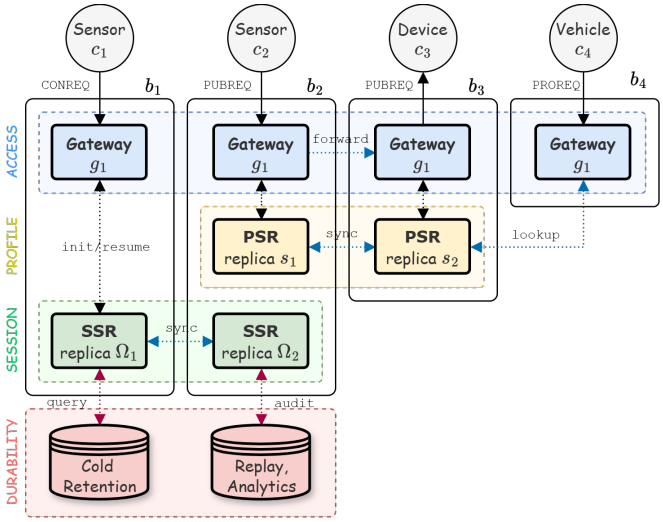


Fig. 18: Logical Planes of the MMQP Hyperscale Fabric

- **Access Plane:** Protocol Gateways (PGs) authenticate clients, validate profiles, manage aliases and QoS, and operate client-facing connections.
- **Profile Shard Plane:** Profile Shard Replicas (PSRs) store versioned session-profile snapshots for their selected shards and answer quorum routing queries.
- **Session Replica Plane:** Session State Replicas (SSRs) store ownership grants, fences, session snapshots, recipient inbox records, and associated recovery state.
- **Durability Plane:** The implemented file store persists bounded broker and cluster records. A retained-descriptor adapter is also available as a reusable component; attaching an external append-only analytics or cold-object service remains an integration option.

(Figure 28) shows the acceptance path for a remote recipient. Profile lookup determines the eligible session, and reliable ingress replicates its inbox before acknowledging acceptance to the publisher. Tree forwarding supplies a prompt notification and data path. The recipient also polls its quorum inbox, so losing a tree branch does not erase work already accepted into that inbox. The recipient's leased owner commits its receipt and outgoing delivery state before sending to the client. Admission to several recipient inboxes is not one distributed atomic transaction. If an operation is interrupted or rejected part-way through fan-out, some recipients may already hold accepted work. Retries preserve the original identity so those recipients can suppress repeated acceptance. The figure describes protocol checkpoints, not a transaction with the subscriber's external business effect.

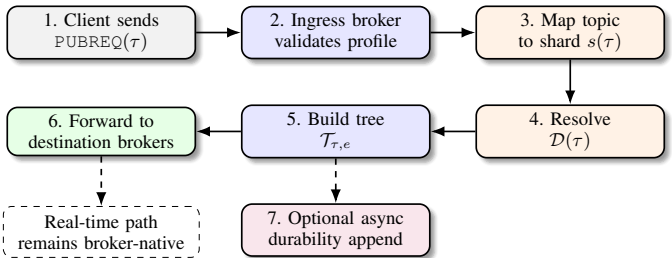


Fig. 19: Reliable Cluster Acceptance and Recovery Path for One Recipient

C. Topic Profile Sharding

Let $\mathcal{B} = \{b_1, \ldots, b_n\}$ be the configured broker set and S the number of logical profile shards available. A concrete canonical topic τ maps to:

$$s(\tau) = H_{64}(\mathrm{UTF8}(\mathrm{canon}(\tau))) \bmod S, \tag{8}$$

where H_{64} is the first eight bytes of SHA-256 interpreted as an unsigned big-endian integer. This specified hash replaces implementation-dependent language-library hashes. Replica placement uses rendezvous ranking. Let $\mathrm{str}_8(x)$ denote a one-octet length followed by the UTF-8 bytes of x. For a shard s, the score assigned to broker b in cluster epoch e is:

$$r_s(b) = H_{64}\big(\texttt{0x01} \parallel \mathrm{BE}_{32}(s) \quad \parallel \mathrm{BE}_{64}(e) \parallel \mathrm{str}_8(b)\big) \tag{9}$$

The shard replica set consists of the R highest-scoring brokers:

$$\mathcal{R}_s = \mathrm{TopK}_R\{(r_s(b), b) : b \in \mathcal{B}\} \tag{10}$$

Tied scores are resolved by ascending broker identifier. Broker-set order in the configuration therefore does not change placement. A wildcard filter cannot be placed by hashing that filter once: its matching concrete topics may hash to every shard. Under the implemented full-topic hash, wildcard assertions are consequently replicated to all logical shards. Concrete profiles retain shard locality, but a population dominated by broad wildcards can approach global profile visibility. The reference implementation stores bounded versioned profile records; a compressed trie or bitmap summary is a possible optimisation, not its current storage representation. For session c and shard s, a live profile record can be represented as:

$$\Sigma_s = \{(S_{id,c}, b_c, f_c, v_c, t_{\mathrm{exp},c}, \mathcal{P}_{c,s})\}, \tag{11}$$

where b_c is the session owner, f_c its fence, v_c its revision, and $\mathcal{P}_{c,s}$ the complete profile contribution visible in that shard.

Quorum reads merge the newest valid (f_c, v_c) record for each session/shard pair. For publication attributes a, the matching destination brokers described as:

$$\mathcal{D}(\tau, a) = \{b_c \mid \exists \pi \in \mathcal{P}_{c,s(\tau)} : \pi \sim \tau \ \wedge \ \text{eligible}(c, \pi, a)\} \tag{12}$$

Eligibility includes QoS, request roles, tenant policy, and shared-group selection. Concrete recipient SIDs are retained alongside their brokers. Routing queries are quorum operations over versioned records, rather than a globally atomic snapshot of every simultaneous subscription change.

D. Deterministic Topic Dissemination Trees

For the selected destination set and ingress broker b_o, define

$$\mathcal{V}_{\tau,e} = \mathcal{D}(\tau, a) \cup \{b_o\}$$

The ingress is *always assigned rank zero*. Remaining distinct destination brokers are ordered by ascending score

$$\kappa_{\tau,e}(b) = H_{64}\big(\text{str}_8(\tau) \parallel \text{BE}_{64}(e) \parallel \text{str}_8(b)\big), \tag{13}$$

with ascending broker identifiers breaking ties. Pinning the ingress is essential: sorting it among all destinations could leave lower-ranked nodes unreachable from the actual source. For fan-out $F \geqslant 1$, the broker at rank $j > 0$ has parent:

$$p(j) = \left\lfloor \frac{j-1}{F} \right\rfloor, \tag{14}$$

and children:

$$\mathcal{C}(b_j) = \{b_k \in \mathcal{V}_{\tau,e} \mid Fj + 1 \leqslant k \leqslant Fj + F\} \tag{15}$$

As illustrated in (Figure 29), brokers with the same accepted recipient set, canonical topic, ingress, epoch, and fan-out reconstruct the same tree. Forwarding records carry the required context, and a receiving broker verifies the authenticated peer's rank and parent relationship before admitting the hop.

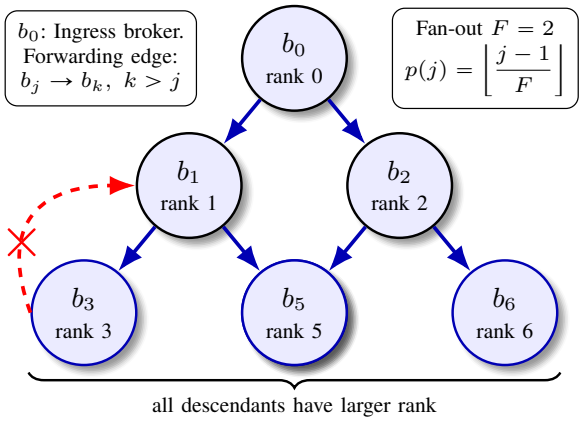


Fig. 20: Deterministic loop-free topic dissemination tree

1) Loop-Freedom Property: Let $d(b_j) = j$ denote tree rank. Every forwarding edge satisfies:

$$b_j \to b_k \quad \Longrightarrow \quad k > j \tag{16}$$

Consequently, along a forwarding path,

$$d(b_{j_0}) < d(b_{j_1}) < \cdots < d(b_{j_h})$$

The destination set is finite, so a valid path cannot revisit a broker. Loop freedom therefore follows from the rank rule for a fixed, validated forwarding record. Duplicate retries may still traverse an edge again; suppressing repeated processing is a separate requirement from preventing a routing cycle.

E. Forwarding Cost Model

Let $M = |\mathcal{D}(\tau, a) \backslash \{b_o\}|$ be the number of remote destination brokers. The tree contains $M + 1$ vertices and M edges, giving an ideal payload-forwarding cost

$$C_{\text{tree}}(\tau) = M$$

For comparison, a direct broadcast from the ingress to all other brokers costs $C_{\text{broadcast}} = n - 1$. For $M > 0$, the corresponding transmission-count ratio is:

$$G(\tau) = \frac{n-1}{M} \tag{17}$$

This comparison uses all-broker broadcast as its baseline; a selectively routed mesh can also avoid uninterested brokers. For $M = 0$, no inter-broker payload forwarding is needed and the ratio is undefined. For the breadth-filled tree and $F \geqslant 2$, the maximum edge depth is:

$$h(\tau) = \left\lceil \log_F\big((F-1)(M+1)+1\big) \right\rceil - 1 \tag{18}$$

For $F = 1$, the tree is a chain and $h = M$. The forwarding component of latency can be approximated by:

$$L_{\text{forward}}(\tau) \approx h(\tau)\, \delta_{bb} + \delta_{ls}, \tag{19}$$

where δ_{bb} denotes a per-hop transport/processing contribution and δ_{ls} local subscriber dispatch. These equations describe tree geometry and an idealised path. The implemented QoS 1 path additionally pays for profile queries, durable inbox replication, owner checkpoints, acknowledgement traffic, and possible recovery polling. In particular, replicated full payloads add traffic beyond the M tree edges. Queueing, storage latency, and shared-link congestion must be measured before drawing end-to-end performance conclusions.

F. Profile Update Propagation

The live binding propagates a complete profile contribution from the current session owner to each affected shard replica set. Each replacement carries the ownership fence, a monotonic revision, and expiry. Removing the last assertion in a shard writes an explicit empty replacement; absence of a new record is not interpreted as deletion. Query merging selects the newest valid version, and expiry compaction removes obsolete

records only after the applicable grace period. Late writes that are already expired are rejected. The framework also contains an independently tested observed-remove profile-set component. It separates assertion and removal-operation dots, tracks contiguous per-origin acknowledgements, and compacts a removal only after every configured replica has acknowledged the relevant removal and observed additions.

G. Session Replication

Session-state replicas are selected by a separate rendezvous domain. For SID $S_{id,c}$, the score is

$$r_c(b) = H_{64}\big(\texttt{0x02} \parallel \mathrm{str}_8(S_{id,c}) \parallel \mathrm{BE}_{64}(e) \parallel \mathrm{str}_8(b)\big),$$

and the selected unit is:

$$\mathcal{U}_c = \mathrm{TopK}_R\{(r_c(b), b) : b \in \mathcal{B}\} \tag{20}$$

A resource operation requires a majority of those replicas:

$$Q_R = \left\lfloor \frac{R}{2} \right\rfloor + 1 \tag{21}$$

The hot access broker maintains the active connection, while replicas preserve profiles, aliases, receive windows and receipts, outgoing identifiers, pending deliveries, fabric receipts, replay queues, and relevant request state. Counting acknowledgements alone is insufficient to establish an owner; the framework adds the following lease and fencing procedure.

1) Ownership, Fencing, and Snapshot Adoption: An owner is identified by its broker identity and a random process incarnation. To acquire a session resource, it reads the current fence, requests a higher fenced grant from a majority, selects the highest accepted snapshot returned by that quorum, and commits adoption of that snapshot to a majority before serving the session. Updates carry the fence and revision. A final ownership check precedes acknowledgements and outgoing session deliveries. A broker that loses its valid majority lease stops serving the session, so a minority partition cannot continue accepting authoritative session updates.

Voters grant a lease of configured duration L. The holder uses a conservative local deadline of $L/3$, measured from sending the request, and cannot renew an already expired local ownership interval back into validity. Restarted voters wait a full lease duration before granting new ownership, because their former monotonic deadlines cannot be restored from disk. The failure model assumes crash/partition faults, unique broker identities, storage that honours successful flushes, and bounded clock-rate drift. The conservative timing margin is intended for ordinary clocks. This is not a Byzantine consensus protocol. Reasonable wall clocks are still needed for message and session expiry, and extended clock/suspend fault testing remains part of further qualification.

2) Recovery and Availability: After the former owner's lease expires, a reachable broker can acquire the session through its selected replica quorum and adopt its snapshot. A surviving client then resumes the same SID, as conceptually illustrated in (Figure 30). The reference implementation in (Section VIII) does not require every access broker to retain every session, but recovery still needs a reachable selected majority, available stored state, and an unexpired session.

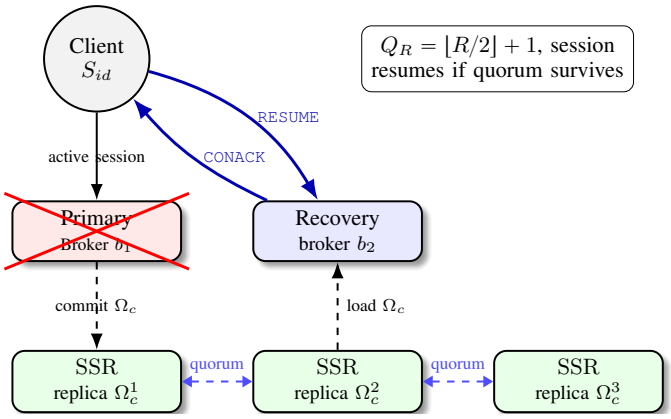


Fig. 21: Session Recovery through the Selected Replica Set

Under an idealised model of independent replica unavailability with probability p_f, the probability that at least Q_R of R replicas remain available is:

$$A_{\text{quorum}} = 1 - \sum_{i=R-Q_R+1}^{R} \binom{R}{i} p_f^i (1-p_f)^{R-i} \tag{22}$$

For $R = 3$ and $Q_R = 2$, this becomes $1 - 3p_f^2(1-p_f) - p_f^3$. This is a quorum-availability calculation under independence, not a measured end-to-end availability result. Correlated failures, network partitions, expired client state, lease-acquisition time, and storage faults can all affect actual recovery.

H. Idempotent Publishing

Inter-broker forwarding uses an immutable Universal Message Identifier (UMID) in addition to the client-facing publication sequence. The identity binds the origin session, original origin sequence, canonical topic, payload hash, and accepted cluster epoch:

$$\begin{aligned} M_{id} = \mathrm{SHA256}\big(&\mathrm{str}_8(\texttt{MHF/1/message}) \parallel \mathrm{str}_8(S_{id}) \\ &\parallel \mathrm{BE}_{64}(P_{\text{origin}}) \parallel \mathrm{SHA256}(p^*) \\ &\parallel \mathrm{str}_8(\tau) \parallel \mathrm{BE}_{64}(e_{\text{accepted}})\big). \end{aligned} \tag{23}$$

Here, p^* is the carried publication payload and all encodings are unambiguous. For client-originated QoS 1, the original publication identifier supplies the origin sequence. Client-originated QoS 0 has no wire PID, so the broker uses a separate persistent counter with its high bit set, keeping that 64-bit sequence distinct from wire identifiers. Broker-generated status traffic uses a separate status origin and a random high-range sequence. The accepted epoch and identity remain unchanged on retry. A future change in routing epoch must not create a new identity for previously accepted work.

Ingress first applies the client-session receipt/window checks from (Section IV-E). For reliable cluster delivery, each selected recipient obtains a durable replicated inbox entry. On consumption, the recipient owner records the fabric identity and outgoing session state before delivery, then writes a consumed tombstone. A lost notification is recovered by reading the inbox again, and a repeated identity does not allocate another outgoing delivery after its receipt has committed. Bounded in-memory epoch filters are supplementary primitives; durable recovery does not depend only on a temporary duplicate cache. These checkpoints support repeatable acceptance across the implemented failover scenarios. They do not make publication fan-out atomic across recipients or establish exactly-once business effects in external applications. QoS 0 remains a transient path even though it receives a fabric identity for authenticated forwarding.

I. Back-Plane Mapping

The live backplane uses a dedicated mutually authenticated QUIC or TLS 1.3 listener. Its private, versioned binding carries `MHF/2` control requests inside MMQP `PUBREQ` envelopes on `"$mhf/control"`; payload transfer uses `"$mhf/data"`. The control envelope contains the `MHF2` magic, 64-bit cluster epoch, one-octet operation kind, 64-bit request identifier, length-prefixed broker identity, 32-bit payload length, payload, and HMAC-SHA256 tag. A HELLO exchange verifies a manifest digest binding the schema, membership, placement parameters, and certificate pins. Native forwarding wraps an authenticated `MHF/1` record together with concrete recipient broker/SID pairs. That record contains the immutable message identity, accepted and routing epochs, origin sequence, expiry, origin session, ingress, sender rank, destinations, and complete concrete-topic publication. Receivers verify the MAC, authenticated sender identity, epoch, and tree-parent relationship before admitting it. Multibyte integers use big-endian encoding, and persisted expiry values use Unix milliseconds. On QUIC, control and data follow the negotiated semantic mapping; eligible transient data can use datagrams, and large records can use bounded bulk streams. The implementation does not create one stream per profile shard or exchange automatic epoch-change messages. The `MHF/1` and `MHF/2` formats are reference bindings filling gaps in the scientific design, with no claim of independent B2B interoperability.

J. Back-Pressure and Admission Control

Let λ_s and μ_s denote the long-run arrival and service rates of a shard-related work queue. The familiar condition

$$\rho_s = \frac{\lambda_s}{\mu_s} < 1 \tag{24}$$

is a useful capacity-planning requirement, but does not itself bound a burst. The implementation uses configured byte and record budgets for packets, pending sends, session snapshots, inboxes, retained messages, replay backlogs, and storage. If a new record consumes $b(m)$ bytes, admission requires, for the relevant queue,

$$Q_s^{\text{bytes}} + b(m) \leqslant B_s, \qquad Q_s^{\text{records}} + 1 \leqslant N_s$$

There is no implemented adaptive formula that derives these limits automatically from measured drain capacity. Reliable work that cannot be admitted receives a failure or times out without a successful acceptance acknowledgement. Best-effort traffic can be lost when its path is unavailable. Bounded retries preserve the original identity, and retained replay waits for outgoing acknowledgement credit instead of placing the entire replay into the in-flight window. This provides explicit overload behaviour, while avoiding unbounded memory growth. Shards still share executors, network paths, and storage devices, so logical sharding alone is not complete performance isolation.

K. Retained Messages and Cold Replay

The live cluster stores retained messages in quorum-owned per-topic queues with a replicated index for discovery. It currently replicates complete payload records, including encrypted envelopes when present. Queue contents follow the publisher's last-value or FIFO policy, message expiry, and configured byte/record limits. Expired records and consumed tombstones are compacted after the applicable expiry/grace conditions; a rejected late expired update cannot recreate them. Under a fixed retention policy, let $A_\tau(t - T_\tau, t]$ count eligible publications in the unexpired interval and let $L_{\max}(\tau)$ be one in last-value mode or the configured FIFO limit. The number of logical retained records for shard s satisfies:

$$N_s^{\text{ret}}(t) \leqslant \sum_{\tau : s(\tau) = s} \min(L_{\max}(\tau), A_\tau(t - T_\tau, t]) \tag{25}$$

This is a count bound; payload sizes, metadata, replica multiplicity, and temporary writes must also fit their byte budgets. Replacing the arrival count with an average rate would provide only a workload estimate, not the implemented hard bound. The reusable durability adapter also supports retained descriptors containing an identity, topic, payload location, expiry, policy, and checksum. The current clustered path does not perform descriptor-only dissemination or fetch cold payloads from an external object store. Those are possible extensions. In the implemented path, a new eligible subscription builds a bounded replay backlog from stored full records, then drains it in order as `PUBACK` releases credit.

L. Failure Handling

Running-cluster membership is static. A manifest binds member identifiers, certificate pins, shard count, replica count, fan-out, schema, and cluster epoch. Startup rejects a conflicting manifest. A broker failure does not automatically advance the epoch or redistribute membership. Changing that configuration requires a planned offline migration and application recovery procedure; arbitrary online shard migration is outside this binding. Within the fixed membership, session failover uses

lease expiry, a higher fence, and quorum snapshot adoption. If a forwarding broker fails, the immediate tree path to its descendants may be interrupted. For a fixed tree, the structural impact at broker b_j is its subtree, and an idealised uniform-failure-position average is:

$$\mathrm{E}[A_f] = \frac{1}{M+1} \sum_{j=0}^{M} |\mathrm{subtree}(b_j)| \tag{26}$$

This quantity describes affected notification paths, not a guarantee that recovery sends only to that subtree. Reliable recipient inboxes are already replicated before successful ingress acknowledgement; their owners recover accepted work through quorum reads even when the tree notification is lost. Transient QoS 0 traffic has no corresponding durable recovery obligation. Whole-cluster restart restores the persistent resource records under the same manifest and observes voter quarantine before granting new leases. A surviving client's resumption then depends on its retained identifiers and unexpired session. A new client process instead creates a new session and can obtain retained application data through a new subscription. Process-termination recovery has been tested; power-loss behaviour still depends on the target filesystem and storage stack honouring flush and replacement operations.

M. Security and Tenant Isolation

Each configured tenant has an authentication identity, a separate authentication key, a namespace root, and an optional allowed-shard set $\mathcal{S}_\theta \subseteq \{0, \ldots, S-1\}$. Admission checks the client's realm, profile role, namespace, and shard policy:

$$\mathrm{authz}(c, \tau, \mathrm{op}) = 1 \quad \wedge \quad s(\tau) \in \mathcal{S}_{\theta(c)}$$

The same scope checks cover registrations, subscriptions, publications, wills, and inbox delivery. A wildcard must be allowed in every shard it can match. All brokers receive the same tenant configuration, and a resumed SID cannot switch realms. This is tenant-level credential and namespace isolation, not an implemented per-device Access Control List (ACL) provider. Each B2B peer presents a distinct certificate that passes CA and hostname verification and matches its configured SHA-256 certificate pin. Control and forwarding envelopes additionally use an epoch-bound shared MAC key of at least 32 bytes:

$$\eta = \mathrm{HMAC\text{-}SHA256}_{K_e}(\text{encoded envelope without tag})$$

The Message Authentication Code (MAC) authenticates an envelope; it is not a digital signature. Epoch checks, request identifiers, fences, revisions, and receipts constrain stale or repeated operations. A compromised authorised broker holding the shared key remains inside the trust boundary and is not neutralised by the MAC. Certificate pins and the storage manifest also make online certificate/epoch rotation an explicit future reconfiguration task.

N. Implemented Properties and Scope

The MHF combines deterministic placement and tree geometry with a concrete ownership and recovery protocol. Its implemented properties can be stated as follows:

- **Routing state:** Concrete profiles are sharded; broad wildcard profiles are replicated across their possible shards. Per-node state depends on assigned replicas and wildcard population.
- **Tree forwarding:** An ingress-pinned tree uses M remote edges and logarithmic depth for $F \geqslant 2$, before accounting for durability and control traffic.
- **Loop prevention:** Validated tree edges increase rank strictly. Receipts separately suppress repeated acceptance during retries and recovery.
- **Session ownership:** A selected majority grants one fenced lease owner, with conservative deadlines, restart quarantine, and snapshot adoption.
- **Reliable recovery:** Persistent inboxes and session checkpoints support the verified node-failure and restart scenarios while preserving the distinction between protocol acceptance and application effects.
- **Deployment boundary:** Membership is static, storage is bounded, and online migration, Byzantine tolerance, and large-scale performance qualification remain outside the current result.

These mechanisms provide a runnable basis for evaluating clustered MMQP under controlled workloads. Their functional validation and software realisation are described next.